

SYS.AGENT // Upgrade Suggestions

Code review of app.py, agent_core.py, assistant.py, sessions.py, and README.md — prioritized list of improvements.

This is a well-structured project already — clean separation between UI, CLI, and shared core, with a genuinely hardened sandbox (capability drops, resource caps, timeouts, redaction). The items below are prioritized to make it better.

Security (highest impact)

1. Docker socket mount is a full host-root escape hatch

docker-compose.yml (line 14) mounts /var/run/docker.sock into the container so it can spawn sandbox containers. Anyone with access to the app (or who compromises it) effectively has root on the host, since Docker socket access equals container escape. If this ever leaves the local machine, this is the #1 thing to fix — e.g. run a sibling "sandbox-runner" daemon with a restricted API instead of raw socket access, or at minimum keep this strictly localhost-only and never expose port 8501 beyond 127.0.0.1.

2. No auth on the Streamlit app

docker-compose.yml publishes 8501:8501 on all interfaces. Anyone on the network who reaches that port can drive an agent that executes arbitrary code in a sandbox with Docker-socket access. Worth binding to 127.0.0.1:8501 or adding basic auth in front (reverse proxy / Streamlit auth) if this ever runs on a shared machine.

3. CLI (assistant.py) never redacts secrets

Sandbox output is never redacted before being printed or fed back into chat_memory (assistant.py, lines 85-92). The Streamlit UI redacts via redact_secrets / redact=True, but the CLI path skips it entirely, so credentials found during recon could leak into terminal scrollbar or get echoed back to the model. Fix: call agent_core.redact_secrets before printing/storing.

4. Sandbox image isn't pinned

SANDBOX_IMAGE = "python:3.11-alpine" (agent_core.py, line 39) floats on whatever the tag currently resolves to; pin a digest (python:3.11-alpine@sha256:...) for reproducibility and to avoid a supply-chain surprise if the tag is ever repointed.

Consistency / Drift Risk

5. assistant.py duplicates logic instead of sharing it

The CLI has its own separate SYSTEM_PROMPT, a single-block extract_code, and no scope/agent-mode support, duplicating logic that agent_core.build_system_prompt / extract_all_code already generalize for the UI (assistant.py lines 25-33; app.py lines 205-212). The README explicitly says both front-ends "share one core" to avoid drift — the CLI has already drifted. Worth having it call the shared prompt builder and multi-block extractor.

6. Workspace path resolution differs between front-ends

app.py uses `os.path.abspath("workspace")` (relative to current working directory) while `assistant.py` / `sessions.py` use paths relative to `__file__`. If Streamlit is ever launched from a different cwd, app.py's workspace silently ends up somewhere else. Standardize on the `__file__`-relative pattern everywhere.

Smaller Improvements

- `audit.log` (`agent_core.py`, line 27) has no rotation/size cap — will grow unbounded over long-running engagements.
- Saved sessions/findings (`sessions.py`, lines 23-34) are plaintext JSON on disk with no encryption — could matter since they may capture live recon data (hosts, hashes, headers) from real engagements.
- `wipe_workspace()` in the sidebar (`app.py`, line 320) is a single click with no confirmation — one misclick destroys all sandbox files.
- `requirements.txt` only pins lower bounds (`>=`) — for a security tool, pinning exact versions (or a lockfile) makes runs reproducible and avoids an unreviewed transitive update changing behavior.
- Default model is explicitly an "abliterated" (uncensored) build — intentional per the README, but worth a loud runtime banner/warning (not just doc text) so nobody forgets the guardrails are prompt-level only, not model-level.

Overall the core design (human-in-the-loop gate, redaction, resource-capped sandbox, audit log) is solid. The Docker-socket exposure and the CLI/UI logic drift are the two to fix first.